

Database - Schema and Migrations

The Prisma schema, the seed, and the five ordered migrations

EdgePilot AI - Team 15
Documentation set, 23 August 2026
7 documents in this volume

prisma/ - database schema, migrations and seed

Work package: integration and API surface. The four newest migrations came out of the benchmark and DevOps work.

What this folder is

The single definition of the database. Table and column names are snake_case in Postgres and camelCase in TypeScript; the @map() directives in the schema do the translation, which is why nothing in the repository layer has to.

That mapping is contractual, not cosmetic. providers.is_active is what /api/v1/providers returns as is_active, so renaming a @map() value without a migration breaks the wire contract as well as the database.

What happened here

Four migrations were added, and one of them removes a table rather than adding one.

The schema grew by 32 lines net (167 to 199), but the shape of the change matters more than the size: benchmarks.device_id and the whole devices table are gone, users.session_id arrived so rows can be owned without accounts, benchmark_results.warmup arrived so a discarded cold-start iteration can be kept and marked rather than deleted, readiness_scores.hardware_fit became nullable so "not measured" stops being reported as a score, and readiness_scores.privacy_class arrived because privacy became an ordinal class instead of a number in an average.

seed.ts shrank from 127 lines to 56 - the device seed data went with the table.

The operational rule, which matters more than any of this: against the shared Neon database, only ever run migrate deploy. Never migrate dev, which can offer to reset and would wipe everyone's data. The db:neon: * scripts in scripts/db/ exist to enforce that.

Files

File	What it is
schema.prisma	199 lines. Seven models: User, Workload, Provider, Benchmark, BenchmarkResult, ReadinessScore, SharedFinding, plus rate_limits. Carries /// doc comments explaining the non-obvious columns
seed.ts	56 lines. The provider catalogue every environment starts with. Safe to re-run: upsert() with an empty update leaves an existing row exactly as it is, so re-seeding never duplicates a provider and never clobbers a deliberate local change
migrations/	Five ordered migrations plus the lock file

Connected folders

- prisma/migrations/ - the ordered history.
- src/lib/ - prisma.ts, the single client for the process.
- src/modules/benchmark/infrastructure/repositories/ - the only code that talks to Prisma for benchmarks.
- scripts/db/ - the guards around the shared database.

- docs/internal/ - database.md, currently stale.

Referenced from

These folders point here. Each link below resolves in both directions.

- /
- prisma/migrations/20260726225537_init/
- prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/
- prisma/migrations/20260822081030_optional_email_and_session_owner/
- prisma/migrations/20260823035500_drop_devices/
- prisma/migrations/20260823165500_warmup_iteration/
- scripts/dev/
- src/
- src/app/api/v1/providers/
- src/app/api/v1/workloads/
- src/modules/benchmark/core/entities/
- src/modules/benchmark/infrastructure/repositories/

prisma/migrations/ - the ordered schema history

Work packages: integration and API surface (the initial migration); benchmark and DevOps (the four newest).

What this folder is

Five timestamped folders, each holding one `migration.sql`, applied in name order. Prisma records which have run; `migration_lock.toml` pins the provider.

What happened here

Four of the five are new. Taken together they are the database half of every change described in 00-PROJECT-RECORD.md, and they were all tested against a local sandbox database first and only then applied to the shared Neon instance with `migrate deploy`.

One is a **relaxing** change and one is a **destructive** change, and the difference is worth knowing before you read them. `DROP NOT NULL` on `hardware_fit` is relaxing - it widens what the column accepts, so it cannot break a running application. `DROP TABLE "devices"` is destructive and irreversible, which is why it was rehearsed locally first.

Files

Migration	What it does
20260726225537_init/	168 lines. The original schema: users, workloads, devices, providers, benchmarks, benchmark_results, readiness_scores, rate_limits, with their indexes and unique constraints
20260822074319_nullable_hardware_fit_and_shared_findings/	Makes <code>readiness_scores.hardware_fit</code> nullable, adds <code>privacy_class</code> , and creates <code>shared_findings</code> with indexes on <code>session_id</code> and <code>created_at</code> . Nullable hardware fit is what lets "not measured" stay null instead of becoming 0
20260822081030_optional_email_and_session_owner/	Adds <code>users.session_id</code> with a unique index and makes email optional. This is what makes account-free row ownership possible
20260823035500_drop_devices/	31 lines. Drops two foreign keys, the <code>benchmarks_device_id_idx</code> index, the <code>benchmarks.device_id</code> column, and the <code>devices</code> table
20260823165500_warmup_iteration/	Adds <code>benchmark_results.warmup</code> <code>BOOLEAN NOT NULL DEFAULT false</code> . The discarded cold-start iteration is stored and flagged, never deleted - the arithmetic can be checked from the rows
migration_lock.toml	Provider lock. Not hand-edited

Connected folders

- prisma/ - the schema these migrations produce.
- scripts/db/ - neon.mjs, the only sanctioned way to run these against the shared database.
- src/modules/benchmark/application/services/ - the runner that writes the warmup flag.

Referenced from

These folders point here. Each link below resolves in both directions.

- prisma/migrations/20260726225537_init/
- prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/
- prisma/migrations/20260822081030_optional_email_and_session_owner/
- prisma/migrations/20260823035500_drop_devices/
- prisma/migrations/20260823165500_warmup_iteration/
- src/app/api/v1/readiness/[id]/ (FOLDER.md)
- src/app/api/v1/session-log/share/

prisma/migrations/20260726225537_init/

prisma/migrations/20260726225537_init/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: the original schema - eight tables, their indexes and unique constraints. Everything else in this folder is a delta against it.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/

prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: makes hardware fit nullable, adds the privacy class column, and creates the shared_findings table.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

prisma/migrations/20260822081030_optional_email_and_session_owner/

prisma/migrations/20260822081030_optional_email_and_session_owner/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: adds a unique session id to users and makes email optional, so a row can be owned without an account.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

prisma/migrations/20260823035500_drop_devices/

prisma/migrations/20260823035500_drop_devices/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: removes the devices table and the benchmarks.device_id column, with their constraints and index.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

prisma/migrations/20260823165500_warmup_iteration/

prisma/migrations/20260823165500_warmup_iteration/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: adds the warmup flag to benchmark_results so a discarded cold-start iteration is marked rather than deleted.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.